Code Explanation

Processing Customer Order Data with Spark

This code is designed to process customer and order data using Apache Spark. It reads data from CSV files, cleans and transforms it, and then loads the processed data into Delta tables.

Overview of the Code
The code is structured into several key steps: creating a Spark session, reading and cleaning customer and order data, joining the data, and loading it into Delta tables. It also creates and populates two specific tables: `ordersummary` and `customeraggregatespend`.

Step 1: Create a Spark Session
This step involves creating a SparkSession, which is the entry point to any Spark functionality. The `create_spark_session` function is used for this purpose.

```
return SparkSession.builder
    .appName("Customer Order Processing")
    .enableHiveSupport()
    .getOrCreate()
```

Step 2: Define Schemas for Customer and Order Data
The schemas for customer and order data are defined using `StructType` and `StructField`. These schemas are crucial for reading the CSV data correctly.

```
customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

order_schema = StructType([
    StructField("OrderId", StringType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", DateType(), True),
    StructField("CustId", StringType(), True)
])
```

Step 3: Read Source CSV Data
The customer and order data are read from CSV files using the defined schemas. The `spark.read.format("csv")` method is used for this purpose.

```
customer_df = (
    spark.read.format("csv")
    .option("header", "true")
    .schema(customer_schema)
    .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdat
a")
)

order_df = (
```

```
    spark.read.format("csv")
    .option("header", "true")
    .schema(order_schema)
    .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata")
)
```

Step 4: Clean Data and Add TotalAmount Column
The customer and order data are cleaned by removing duplicates and filtering out rows with null values. The `TotalAmount` column is added to the order data by multiplying `PricePerUnit` and `Qty`.

```
customer_clean_df = (
    customer_df
    .dropDuplicates()
    .filter(
        (F.col("CustId").isNotNull()) &
        (F.col("Name").isNotNull()) &
        (F.col("EmailId").isNotNull()) &
        (F.col("Region").isNotNull())
    )
)

order_clean_df = (
    order_df
    .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
    .dropDuplicates()
    .filter(
        (F.col("OrderId").isNotNull()) &
        (F.col("ItemName").isNotNull()) &
        (F.col("PricePerUnit").isNotNull()) &
        (F.col("Qty").isNotNull()) &
        (F.col("Date").isNotNull()) &
        (F.col("CustId").isNotNull())
    )
)
```

Step 5: Create ordersummary Table if Not Exists
The `ordersummary` table is created if it does not exist. The table is defined with the required columns and is partitioned by the `Date` column.

```
spark.sql(f"""
CREATE TABLE IF NOT EXISTS gen_ai_poc_databrickscoe.sdlc_wizard.orde
rsummary (
    CustId STRING,
    Name STRING,
    EmailId STRING,
    Region STRING,
    OrderId STRING,
    ItemName STRING,
    PricePerUnit DOUBLE,
    Qty INT,
    Date DATE,
    IsActive BOOLEAN,
    StartDate TIMESTAMP,
    EndDate TIMESTAMP
)
USING DELTA
```

```
PARTITIONED BY (Date)
""")
```

Step 6: Join Customer and Order Data
The cleaned customer and order data are joined on the `CustId` column using an inner join.

```
joined_df = (
    order_clean_df
    .join(
        customer_clean_df,
        on="CustId",
        how="inner"
    )
    .select(
        "CustId", "Name", "EmailId", "Region", "OrderId",
        "ItemName", "PricePerUnit", "Qty", "Date"
    )
)
```

Step 7: Add IsActive, StartDate, and EndDate Columns
The `IsActive`, `StartDate`, and `EndDate` columns are added to the joined data. The `IsActive` column is set to `True`, `StartDate` is set to the current timestamp, and `EndDate` is set to `None`.

```
joined_df = (
    joined_df
    .withColumn("IsActive", F.lit(True))
    .withColumn("StartDate", F.current_timestamp())
    .withColumn("EndDate", F.lit(None).cast(TimestampType()))
)
```

Step 8: Write to ordersummary Table
The joined data is written to the `ordersummary` table in append mode.

```
joined_df.write.format("delta").mode("append").saveAsTable("gen_ai_p
oc_databrickscoe.sdlc_wizard.ordersummary")
```

Step 9: Create customeraggregatespend Table if Not Exists
The `customeraggregatespend` table is created if it does not exist. The table is defined with the required columns.

```
spark.sql("""
CREATE TABLE IF NOT EXISTS gen_ai_poc_databrickscoe.sdlc_wizard.cust
omeraggregatespend (
    Name STRING,
    TotalAmount DOUBLE,
    Date DATE
)
USING DELTA
""")
```

Step 10: Aggregate and Load Data into customeraggregatespend Table
The order data is aggregated by `Name` and `Date`, and the total amount is calculated. The aggregated data is then written to the `customeraggregatespend` table in overwrite mode.

```
agg_df = (
    order_clean_df
    .join(
        customer_clean_df,
        on="CustId",
        how="inner"
    )
    .groupBy("Name", "Date")
    .agg(F.sum("TotalAmount").alias("TotalAmount"))
)

agg_df.write.format("delta").mode("overwrite").saveAsTable("gen_ai_p
oc_databrickscoe.sdlc_wizard.customeraggregatespend")
```